

LINQ Repeat Method in C# with Examples

In this article, I will discuss the **LINQ Repeat Method in C#** with Examples. Please read our previous article discussing the [LINQ Range Method in C#](#) with Examples. The Repeat method belongs to the Generation Operator category. Generation Operators allow us to create some specific type of Array, Sequence, or Collection using a single expression instead of creating them manually and populating them using some loop.

LINQ Repeat Method in C#:

In C#, the Repeat method is a part of LINQ (Language Integrated Query) and resides in the **System.Linq** namespace. It is used to generate a sequence that contains one repeated value over a specified number of iterations. The LINQ Repeat Method in C# generates a sequence or a collection with a specified number of elements, each containing the same value. The following is the signature of the LINQ Repeat Method.

```
public static IEnumerable<TResult> Repeat<TResult>(TResult element, int count);
```

The Repeat method takes 2 integer parameters. The first parameter (i.e., **TResult element**) specifies the value to be repeated. The second parameter (i.e., **int count**) specifies the number of times to repeat the value. The return type of this method is **IEnumerable<TResult>**, which will contain the repeated values. Here, **TResult** specifies the data type of the value that will be repeated in the result sequence. When the count value is less than 0, it throws **ArgumentOutOfRangeException**.

- **element:** The value to be repeated.
- **count:** The number of times to repeat the value.

Example to Understand LINQ Repeat Method in C#:

Let us see an example of the LINQ Repeat Method in C#. The following example shows how to use the Repeat method to generate a sequence of repeated values. In the example below, we repeat the string **"Welcome to DOT NET Tutorials"** 10 times using the Repeat method. As the data type of the value is a string, it will return **IEnumerable<string>**. Here, **TResult** is a string. No operator call repeat is available in LINQ to write the Query Syntax. So, it only supports the Method Syntax.

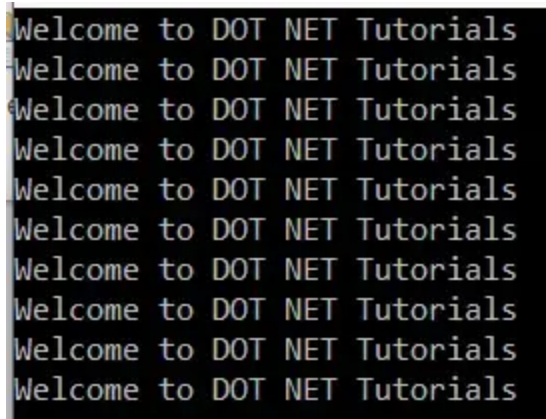
```
using System.Linq;
using System;
using System.Collections.Generic;
namespace LINQRepeatMethodDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Repeating the string value Welcome to DOT NET Tutorials for 10 1
            //Using the Repeat Method
```

```

        IEnumerable<string> repeatStrings = Enumerable.Repeat("Welcome to
Tutorials", 10);
        //Accessing the collection or sequence using a foreach loop
        foreach (string str in repeatStrings)
        {
            Console.WriteLine(str);
        }
        Console.ReadKey();
    }
}

```

Output:



```

Welcome to DOT NET Tutorials
Welcome to DOT NET Tutorials
Welcome to DOT NET Tutorials
Welcome to DOT NET Tutorials
Welcome to DOT NET Tutorials
Welcome to DOT NET Tutorials
Welcome to DOT NET Tutorials
Welcome to DOT NET Tutorials
Welcome to DOT NET Tutorials
Welcome to DOT NET Tutorials

```

What happens if we pass the count as a Negative Number to the Repeat Method in C#?

If we pass the count value as a Negative Number to the Repeat Method, it will throw **ArgumentOutOfRangeException**. For a better understanding, please have a look at the following example. Here, we are passing the count value as -5.

```

using System.Linq;
using System;
using System.Collections.Generic;
namespace LINQRepeatMethodDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Repeating the string value Welcome to DOT NET Tutorials for 10 1
            //Using the Repeat Method
            IEnumerable<string> repeatStrings =
                Enumerable.Repeat("Welcome to DOT NET Tutorials", -5);
            //Accessing the collection or sequence using a foreach loop
            foreach (string str in repeatStrings)
            {

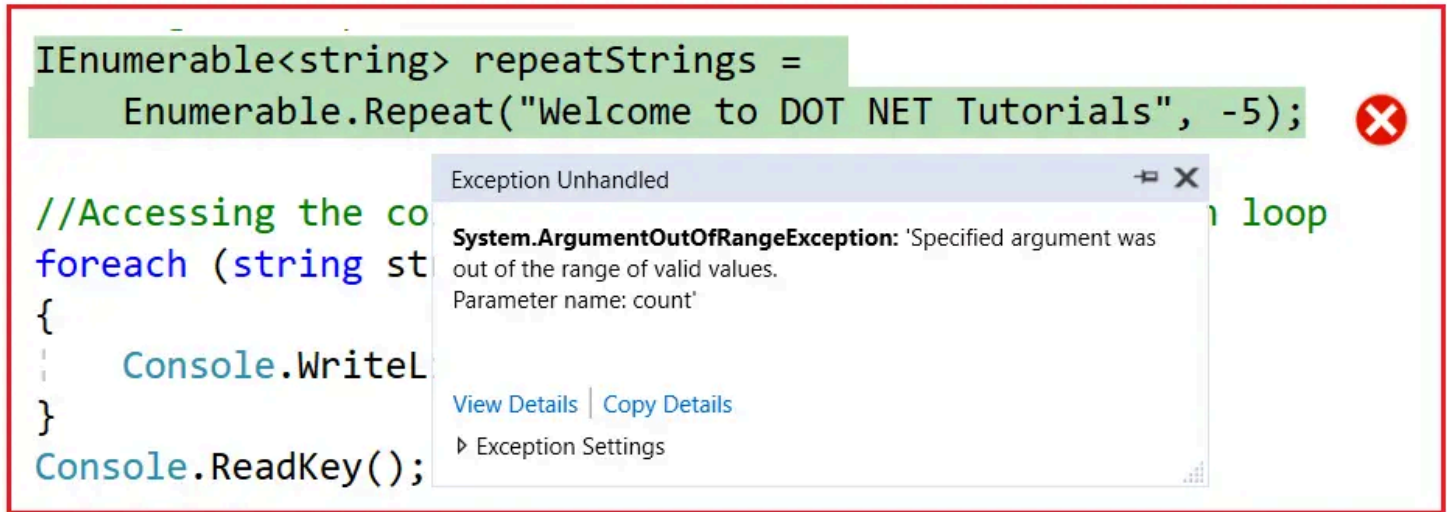
```

```

        Console.WriteLine(str);
    }
    Console.ReadKey();
}
}
}

```

With the above changes in place, now run the application, and you will get the **ArgumentOutOfRangeException** as shown in the below image.



Note: The Repeat method is implemented using the Deferred Execution. So, the immediate return value is an object which stores all the required information to perform an action. The query represented by this method is not executed until the object is enumerated by calling its GetEnumerator method directly or using a for each loop.

When to use the LINQ Repeat Method in C#?

The Enumerable.Repeat method in C# LINQ is handy when generating a sequence of repeated values of the same element. It's particularly useful when creating collections or sequences with consistent or default values. Here are some common situations where you might use the Repeat method:

Initializing Collections with Default Values:

Repeat can initialize arrays, lists, or other collection types with default or initial values. For example, creating an array of zeros or a list of empty strings.

```
int[] zeros = Enumerable.Repeat(0, 10).ToArray();
```

Use Cases:

- **Filling:** When you need to initialize a collection with a default or initial value, Repeat can create a list with the same value for each element, which can then be modified.
- **Placeholder Elements:** Creating a sequence with placeholder values that will be replaced later in the code.

Generating Test Data:

When testing your code, you might need sample data with repeated values. Repeat allows you to create mock data easily.

```
var mockData = Enumerable.Repeat(new TestData(), 100).ToList();
```

Use Cases:

- **Testing:** For unit tests, you must create a collection of items with the same value to test bulk operations or performance.
- **Mock Data:** Generating mock data for testing or design purposes, such as populating a UI control with dummy data to see how it looks.

Creating Sequences for Join Operations:

You might need to create sequences to join with other data sources in database operations or LINQ queries. Repeat can help you create sequences that match the length or count of other collections.

```
var keys = Enumerable.Range(1, 10);  
var values = Enumerable.Repeat("Default", 10);  
var joinedData = keys.Zip(values, (k, v) => new { Key = k, Value = v });
```

Populating Grids or Tables with Default Values:

In UI development or data visualization, you might want to initialize grids, tables, or matrices with default values.

```
var grid = new int[5, 5];  
for (var i = 0; i < 5; i++)  
{  
    grid[i, i] = 1;  
}
```

Padding Sequences:

Repeat can be useful When working with data sequences; you need to pad them with a specific value to meet a certain length or structure.

```
var data = new List<int> { 1, 2, 3 };  
data.AddRange(Enumerable.Repeat(0, 5 - data.Count));
```

Setting Default Values in Aggregations:

When performing aggregations, you can use Repeat to set default values for missing or empty elements in your data.

```
var data = new List<int> { 1, 2, 3 };  
var sum = data.Concat(Enumerable.Repeat(0, 5 - data.Count)).Sum();
```

So, the Repeat method in C# LINQ is useful when you need to generate sequences of repeated values, especially for initializing collections, creating test data, padding sequences, and other scenarios where consistent or default values are required. It simplifies generating such sequences and can make your code more concise and readable.

In the next article, I will discuss the [LINQ Empty Method in C#](#) with Examples. In this article, I explain the **LINQ Repeat Method** in C# with an Example.